Pluses: An interesting research question, asking how much temperature-specific data a model needs to transfer. Leave-one-cell-out validation is a sensible choice. The ethics section stands out, particularly the discussion of manufacturers keeping field data closed and the commitment to report uncertainty and validated ranges. The weekly milestone plan was helpful.
Minuses: The motivation would be more credible if it were backed by some numbers and evidence. Only one ML model was mentioned with no details on evaluation metrics, a baseline, or a success target. Testing more models would enable comparison and identification of the most suitable model. The NASA dataset may have limited data on cells at each temperature, so it's worth checking how many are usable and what that means for the project. Technical choices need to be justified with evidence or reasoning rather than referring to the responses from an AI tool. It's also worth clarifying whether the model predicts capacity (SoH) or drive time, as the abstract and objectives differ.
For now complete all of the code to run all of the models so that I can run it on my 9070xt at home, create multiple models that we can test and choose the best models from. https://zenodo.org/records/6405084 just to be clear you are using data set 1 from this link right, if not fix it to run on this dataset

Worked for 9m 32s
Continue this chat
Worked for 9m 11s

Completed the corrected package: Battery_Model_Lab_Dataset1.zip.

It now uses **Dataset 1 from your Zenodo link**, with 66 cells and 22,239 usable cycles.

Extract the ZIP, then double-click a numbered model launcher. Each of the eight models runs separately and saves its training history, metrics, graphs and discussion notes. Use `09_VIEW_COMPARISON.bat` to compare results.

All eight passed short CPU integration tests; Ridge also passed a full-data run. RX 9070 XT execution remains untested here—the launcher attempts AMD acceleration and provides CPU fallback.

Is this what this graph for the Run_mean model
Worked for 13s

Yes—this is exactly the expected shape for `RUN_MEAN`. It indicates the baseline is behaving as intended.

The mean model ignores the input features and predicts the **average capacity of its training data**, approximately **2.95 Ah**, for every test cycle. That produces the horizontal band.

- **Dashed diagonal:** perfect predictions, where predicted capacity equals measured capacity.

- **Coloured points:** predictions for each held-out battery.
- **Small differences between horizontal bands:** leaving out a different battery slightly changes the training average.

It therefore **overestimates low-capacity cells and underestimates high-capacity cells**. This is intentional: the other models should demonstrate that learning from the voltage-relaxation features improves on this simple baseline.

For your report, you could write:

> The mean baseline produced approximately constant predictions of 2.95 Ah and could not capture changes in battery capacity. It provides a reference against which the predictive benefit of the other models can be assessed using held-out-cell errors.

could you make a shap feature importance for each temeprature
Worked for 5m 23s

Yes—added SHAP plots for **25°C, 35°C and 45°C** for each completed model.

Battery_Model_Lab_Dataset1_SHAP.zip

1. Extract the updated package.
2. Copy your existing **`results` folder** into its folder.
3. Double-click **`10_SHAP_BY_TEMPERATURE.bat`**.

It generates feature-importance bars, signed SHAP plots, temperature comparisons and CSVs—**no retraining required**.

These explain your saved **25°C-trained model** at each temperature, before target-temperature retraining. For `RUN_MEAN`, every feature's importance will correctly be zero.

Verified on all eight model families using CPU.

can I just put the number 10 executable into the original file as this error has occurred
Worked for 10s

Yes—use your original working folder, but copy these four files from the updated package, not just number 10:

- `10_SHAP_BY_TEMPERATURE.bat`
- `shap_temperature.py`
- `requirements_shap.txt`
- `launch.ps1` — choose **Replace the file** when prompted.

Put them alongside your original numbered launchers. **Keep your existing `.venv`, `data` and `results` folders.** Then double-click `10_SHAP_BY_TEMPERATURE.bat`.

The screenshot shows that the new folder tried to install Python but couldn't find the resulting executable. Using your original folder should reuse the Python environment that already ran your models.

You do not need to retrain anything.

Is this report completely optimised to give the best possible chance of receiving 100%

It is a strong, rubric-aligned report, but I would not honestly call it completely optimised or guarantee 100%. Addressing every criterion is necessary; full marks also depend on the strength of the evidence, clarity of interpretation and the marker's judgement.

The most important remaining considerations are:

Area What could affect full marks

Experimental evidence Only three cells represent 35°C, and the experiments use a single random seed. These limitations are discussed, but stronger evidence would require additional experiments.

SHAP interpretation The explanations concern models trained at 25°C. They cannot establish physical temperature effects or explain the behaviour of the retrained models at every temperature.

Model selection The small MLP and SVR have similar source-temperature performance. The report appropriately avoids claiming a decisive winner where uncertainty overlaps.

Project timeline The testing sequence is supported by run records, but the reasons are a retrospective interpretation. Your team should verify that these accurately describe its decisions.

Submission requirements Your requested section allocations total 11 pages, before the cover and references. Check this against any formal page limit, particularly the earlier "about 10 pages" requirement.

For the existing results, the report gives you a strong submission. The biggest remaining improvement would be further validation—particularly repeated neural-network runs and testing the removal of the temperature feature—rather than simply making the writing sound more confident.

Before submitting, add the full names and student IDs, confirm the team's account of the work, and check the formal page limit and any required AI-use declaration.

Make sure that the retrospective project timeline is actually a real timeline and isnt retrospective in a word document and do anything possible that would give better marks. If there is something else that needs to be compelted that you cannot do in one turn advise me

Updated the report with actual logged start and finish times for all eight models on 22 October, without inventing earlier milestones or team decisions. I also strengthened the analysis with individual-cell success rates and clearer limits on the recommendations.

Further improvements requiring additional work: repeated-seed training and constant-feature removal experiments need the raw dataset or complete extracted feature table. Independent confirmation requires new cells. These are documented as future work, not completed experiments.

Before submission, add full names and student IDs and confirm that the 11-page body meets the formal page limit.

ENGG2112_Final_Report.docx

Word

ENGG2112_Recorded_Timeline.docx

Word

Further improvements requiring additional work: repeated-seed training and constant-feature removal experiments need the raw dataset or complete extracted feature table. Independent confirmation requires new cells. how do I do all of this

You can do the first two improvements using your existing dataset. Independent confirmation is different: it needs battery cells that have not already influenced model selection. You do not need to collect new data before improving the current report.

I checked your saved training code: it currently fixes the random seed at 42 and does not have a constant-feature-removal option. Simply running the existing executables again would therefore not perform these experiments.

1. Locate your prepared dataset

On your PC, open your original Battery_Model_Lab_Dataset1 project folder and find the data folder. It should contain:

features.csv

feature_names.json

manifest.json

Zip and upload the entire data folder. These files contain the inputs needed for further training; the results.zip you previously uploaded contains outputs and code snapshots.

If that folder is missing, the original Dataset_1_NCA_battery.zip can be processed again using your existing prepare_data.py.

2. Repeat training with different random seeds

A seed controls random choices such as neural-network initialisation and training order. This experiment checks whether the result was reliable or depended on one favourable run.

Use this design:

Setting	What to do
Models	Small MLP and large MLP
Seeds	42, 43, 44, 45 and 46
Dataset and cell partitions	Keep identical

Hyperparameter search	Keep the same procedure and search space
Evaluation	Repeat source validation and temperature-transfer experiments
Reporting	Show mean and standard deviation across seeds, alongside individual results

SVR is deterministic under your current setup, so repeating it with different seed numbers should not add meaningful evidence.

Do not select and report only the best seed. The useful finding is whether the ranking and the 5% calibration result remain consistent.

3. Test removal of constant features

Your original source training uses only 25°C cells. Temperature therefore contains no variation during source training, even though its value changes during testing at 35°C and 45°C.

Compare two otherwise identical pipelines:

Original pipeline	Modified pipeline
Uses all existing predictors	Removes predictors with zero variance in the current training partition
Imputation → scaling → model	Imputation → constant-feature removal → scaling → model

The feature-removal rule must be fitted inside each training fold, without inspecting validation or test cells. Leave the original CSV unchanged.

An important distinction: when source and target donor data are pooled, temperature varies and may remain in the modified pipeline. That is correct—the rule removes features that are constant in that particular training partition.

Run the modified pipeline for both MLPs across the same five seeds, plus SVR once. Compare:

Source-temperature MAE.

Zero-label transfer MAE at 35°C and 45°C.

Transfer performance with 5% donor data.

Worst-cell MAE and the number of cells meeting the 0.05 Ah target.

This tests whether the problem identified through SHAP actually changes prediction performance. An improvement is not guaranteed; either outcome is useful evidence.

4. Understand what independent confirmation requires

Your leave-one-cell-out tests already measure performance on cells excluded from each fit. However, you have now examined those results to choose models and calibration budgets.

A stronger final confirmation would:

Fix the model, preprocessing and calibration budget.

Obtain additional compatible cells whose results have not informed those choices.

Evaluate once without adjusting the method after seeing their errors.

Changing the seed, reshuffling existing cells or repeating leave-one-cell-out validation does not create new independent data. Another chemistry would test a different, broader transfer question.

For this assignment, it is reasonable to retain genuinely independent confirmation as future work if suitable new data are unavailable.

The next practical step is to upload your zipped data folder. With that, the follow-up experiments can be set up as separate launchers with automatic comparison tables,

preserving your original results. The most valuable immediate additions are repeated-seed testing and the constant-feature experiment—not collecting new batteries.

data.zip

File

here is the zip file, I also want you to generate me something that can run all of the training from one executable so that I can just leave my pc running to completely finish everything

Continue this chat

Download the complete training package

To run everything:

Extract the ZIP into a new folder.

Double-click START_ALL_TRAINING.cmd.

Leave the window open until it reports completion.

Your data is included. The launcher automatically sets up dependencies and runs:

32 training experiments, including five seeds for both neural networks.

Original versus constant-feature-removal comparisons.

Temperature-transfer tests and SHAP graphs.

Combined results tables and graphs.

It uses CPU, prevents automatic sleep while running, and skips completed jobs if restarted. Allow several hours and internet access for initial setup.

When finished, upload RESULTS_TO_UPLOAD.zip so I can incorporate the findings into your report.

The training and explanation paths passed smoke tests here; the Windows setup itself still needs to execute on your PC.

Installing Python 3.12.10 for this user from python.org...

The term 'C:\Users\pc gaming\AppData\Local\BatteryValidationPython312\python.exe' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.

Stopped or incomplete. Read the error above. Relaunch to retry unfinished jobs.

Press any key to continue . . . I received this error message when trying to start the cmd file

That was a bug in my setup script—it assumed Python existed at a particular path without verifying it. Your data is unaffected.

Download the corrected training package

Extract it over your existing Battery_Validation_Suite folder.

Choose Replace files when prompted.

Double-click START_ALL_TRAINING.cmd again.

The updated launcher searches for and verifies your Python installation. You don’t need to uninstall anything.

If setup still fails, upload SETUP_LOG.txt from the suite folder so I can diagnose the exact failure.